Treasure Hunt

Names/IDs:

1. Adham Ashraf Nagy 23101364
2. Ahmed Mohamed Ahmed 23101359
3. Ahmed Fekry Abedllatif 23101358
4. AbdelRahman Ahmed Mostafa 23101413
5. Mohamed Rafaat 23100181
6. Muhammed Hany Hamdy 23102359

Introduction

Overview:

A grid-based game using the Pygame library to visualize the game. It uses the A* search algorithm to navigate through obstacles, collect gold, and take damage.

Objective:

The main objective of this game is to find the most efficient path using the A* algorithm, considering tile costs like damage and gold, while minimizing the damage taken and maximizing gold collected to reach all the treasures.

Problem Statement:

The challenge is to navigate a grid from a start position (top-left corner tile) to many goals (treasures) while considering different types of grid cells:

- **Walkable cells:** These cells are free to traverse, assigned to 0.
- **Obstacle cells:** These cells are blocked and cannot be passed through, assigned to x.
- **Damage cells:** These cells cause a loss of health (HP) upon traversal, assigned to 2.
- **Gold cells:** These cells increase the player's gold when visited, assigned to 3.
- **Treasure cells:** These cells are the goals that the player tries to reach and are assigned to 4.

The pathfinding algorithm must find the shortest and most efficient path while considering the unique properties of each tile type. This problem is modeled in a grid of size 20x20, and the algorithm must dynamically update the player's health and gold stats as the path is traversed.

Methodology:

1. **Grid Generation (make_grid function):**
    - The grid is 20x20. Its tiles are initially set to walkable tiles. Then, using the randint function from the random library, obstacles, damage, and gold tiles are randomly placed in the grid. The top-left tile (start tile) and treasure tiles are also placed randomly.



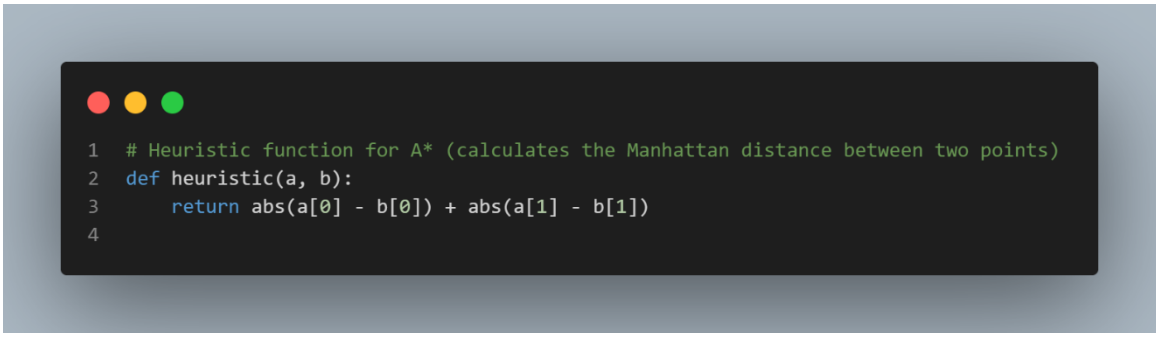
```python
def make_grid():
    grid = [[walkable for _ in range(grid_size)] for _ in range(grid_size)]
    for row in range(grid_size):
        for col in range(grid_size):
            random_value = randint(1, 10)
            if random_value ==1:
                grid[row][col] = obstacle
            elif random_value in (2, 4,5, 9):
                grid[row][col] = damage
            elif random_value ==3:
                grid[row][col] = gold

    # Set start and goal positions as walkable
    grid[0][0] = walkable
    grid[grid_size - 1][grid_size - 1] = walkable

    # Randomly place multiple treasure tiles aka goals
    treasurs_count = 5 # Number of treasure tiles
    for _ in range(treasurs_count):
        treasure_x, treasure_y = randint(0, grid_size - 1), randint(0, grid_size - 1)
        while grid[treasure_y][treasure_x] != walkable: #make sure that the tile type is walkable so the player can move to it
            treasure_x, treasure_y = randint(0, grid_size - 1), randint(0, grid_size - 1)
        grid[treasure_y][treasure_x] = treasure

    return grid
```

2. **Heuristic Function:**
    - The Manhattan distance is used as the heuristic function, which calculates the sum of the horizontal and vertical distances between the current tile and the goal tile. This heuristic is chosen because movement is restricted to the four cardinal directions (up, down, left, right). The Manhattan distance is a good choice for this type of game.



```python
# Heuristic function for A* (calculates the Manhattan distance between two points)
def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

```

3. *A Implementation:**
    - **Open Set:** A priority queue (min-heap) is used to store the nodes to be explored, using the heapq library. Nodes with the lowest estimated cost are explored first.
    - **Visited Set:** A dictionary is used to store visited nodes, avoiding revisiting them.
    - **Cost Calculation:** Each node has a `g` cost (the cost to reach that node from the start) and an `f` cost (sum of `g` and the heuristic). The A* algorithm favors paths with the least `f` cost.
    - **Tile Types Handling:** At each tile reached, the algorithm selects the tile with the least cost while avoiding damage tiles as much as possible and skipping any obstacle tiles.

    Tile Costing:

    - Walkable tiles have a cost of 1.
    - Damage tiles have a high cost (10) so that A* tries to avoid them.
    - Gold tiles have a low cost (0.5) so that A* favors them.
    - Obstacles are skipped by the algorithm.

    Tile Color Function: After creating the grid, each tile is colored according to its type:

    - Black for obstacle tiles.
    - Yellow for gold tiles.
    - Blue for the start and goal tiles.
    - Red for damage tiles.
    - Green for the tiles in the path generated by A*, which are the tiles saved in the traversed set.
4. **Dynamic Player Stats:** As the player traverses the grid, health (HP), initially set to 100, and gold, initially set to 10, are updated based on the tile type:
    - Damage tiles decrease HP by 10.
    - Gold tiles increase the gold amount by 10.
    - Treasure count also increases if the player collects any treasure tiles.

    *Incorporating Stats into A:**

    - As the A* algorithm navigates the grid, it updates the player's health and gold stats dynamically, factoring in the current tile's impact. For instance, the algorithm will try to minimize paths through damage tiles, and maximize paths that pass through gold tiles, although gold collection may not always take precedence if it leads to excessive damage or a longer path to the next treasure.

5. **Visualization:** The grid, path, and player stats are visualized using Pygame, with each tile color-coded according to its type (walkable, obstacle, damage, gold, treasure, or traversed).
6. **Main Function:** In the main function, Pygame is initialized and all the necessary attributes are declared and initialized. When the game window is closed, the game stops running, and the final HP, treasure count, and gold stats are printed to the console.
7. **Limitations and Observed Issues:**
    - As the tiles are randomly generated, the player may reach a position where all the tiles around them are obstacles. If this happens, the algorithm must either reset the game or rerun the grid generation.
    - To address this, a retry mechanism could be added to regenerate the grid if the starting or goal areas are surrounded by obstacles.
8. **Game Over:**
    - The game ends if the player reaches all the goals (treasures) or if their HP becomes zero. The A* algorithm adjusts the path dynamically to ensure optimal paths are taken, but if HP runs out, the game ends with a loss. The player also wins if they reach all the treasures.

Roles:

1. **Adham Ashraf Nagy (23101364):** Researched Pygame and its functions and contributed to developing the A* function and main function.
2. **AbdelRahman Ahmed Mostafa (23101413):** Contributed to developing the A* function.
3. **Ahmed Fekry Abedllatif (23101358):** Managed all the visuals and coloring, developed the color function.
4. **Mohamed Hany Hamdy (23102359):** Researched the random library and used it in developing the make_grid function.
5. **Mohamed Rafaat (23100181):** Searched for the logic behind the heuristic function for the A* implementation.
6. **Ahmed Saleh (23101359):** Handled the main function and ensured the smooth and correct running of the game.

Results:

The implementation successfully computes the optimal path considering tile costs. As the player follows the path:

- The path is highlighted in green, and obstacles are shown in black.
- Damage tiles are in red, gold tiles are in yellow, and treasure tiles are blue.
- The player's HP decreases when stepping on damage tiles and increases gold when collecting gold tiles.

- The algorithm dynamically updates the player's stats and path, with the goal being to maximize the treasures (goals) reached while taking the least possible damage and cost.

Conclusion:

The A* algorithm efficiently finds the best path in a grid-based environment, taking into account various factors such as tile type and costs. The implementation in this project successfully simulates a navigation system where the player must consider both obstacles and advantageous tiles (gold) while avoiding harmful ones (damage). Future improvements could include adding more tile types, such as power-ups, and implementing a more complex cost model. Additionally, optimization techniques could be explored to handle larger grids more efficiently. Introducing a difficulty setting with varying tile types and player stats might further enhance the gameplay.

References:

1. **Pygame Documentation:** https://www.pygame.org/docs/
2. *A Pathfinding Game Tutorial:** https://youtu.be/W9zSr9jnoqY?si=Jj3obn2Ux4HtcmjD
3. **Python Official Documentation:** https://docs.python.org/3/library/random.html
4. **Geeks for Geeks Tutorial:** https://www.geeksforgeeks.org/pygame-tutorial/